The Transformer and Large Language Models

From the Chain Rule to ChatGPT: The Mathematics Behind AI

John Mitchell · Clark College · WAMATYC 2026

May 2026

The Problem with Feedforward Networks on Text

A feedforward network maps a fixed-size input to an output. Every input position has its own weights, and those weights do not interact across positions. This works well for structured tabular data, but language resists it for two reasons.

First, sentences have variable length. A feedforward network needs a fixed input dimension; text does not have one. Second, and more fundamentally, meaning in language depends on relationships between words that can be arbitrarily far apart:

*"The cat, which had been sleeping on the windowsill all morning, finally stretched and yawned. **It** knocked over the plant."*

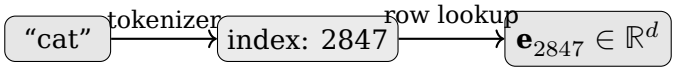
Understanding what *it* refers to requires connecting it to *cat* across many intervening tokens. A fixed-weight feedforward layer has no mechanism for this — each position's computation is independent of every other. The transformer was designed specifically to solve this problem.

Historical note. The transformer architecture was introduced by Vaswani et al. in the paper "Attention Is All You Need" (2017). Prior to this, sequence modeling relied on recurrent neural networks (RNNs) and their variants (LSTMs, GRUs), which processed tokens sequentially and struggled to maintain information across long sequences. The transformer replaced recurrence entirely with attention, enabling parallelization over the full sequence and dramatically improving performance on long-range dependencies. GPT, BERT, and every major language model since 2018 is built on this architecture.

Embeddings: Words as Vectors

Neural networks operate on numbers, not words. The first step in any language model is converting tokens to vectors.

A *vocabulary* of size V assigns each word (or subword) an integer index. An *embedding matrix* $E \in \mathbb{R}^{V \times d}$ maps each index to a dense vector of dimension d:



The embedding of token i is simply row i of E. The lookup is not a computation — it is an indexing operation.

How Embeddings Are Learned

The embedding matrix E is initialized randomly and updated by gradient descent during training, exactly like every other parameter matrix in the network. There is no separate embedding training step. The model learns embeddings that are useful for predicting the next token, and in doing so discovers structure: words with similar meanings end up with similar embedding vectors, and arithmetic relationships emerge (the classic example: $\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}$).

The embedding matrix for GPT-3 has vocabulary size $V = 50{,}257$ and embedding dimension $d = 12{,}288$, giving roughly 617 million parameters in this one matrix alone.

Key result. Embeddings are not hand-crafted feature vectors. They are learned parameters, updated by the same gradient descent mechanism as the rest of the network. The geometric structure in embedding space — similarity, analogies, clusters — emerges from the training objective, not from explicit design.

Positional Encoding

The embedding of a token carries its meaning but not its position. The word "not'' has the same embedding whether it appears first or last in a sentence — but its position changes the meaning entirely.

Transformers solve this by adding a *positional encoding* vector $\mathbf{p}_t$ to each token embedding before processing:

$$\mathbf{x}_t = \mathbf{e}_t + \mathbf{p}_t$$

The original transformer used fixed sinusoidal encodings:

$$p_{t,2k} = \sin\left(\frac{t}{10000^{2k/d}}\right) \qquad p_{t,2k+1} = \cos\left(\frac{t}{10000^{2k/d}}\right)$$

Modern models typically use learned positional embeddings — a second matrix of size $T_{\text{max}} \times d$ trained alongside E. Either way, position information enters the model as an additive signal on the token vectors before any attention is computed.

The Attention Idea

The central question attention asks for each token is: *which other tokens in this sequence are relevant to understanding me?*

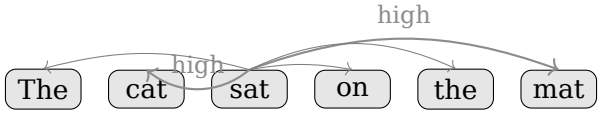
Consider the sentence `The cat sat on the mat.'' When the model processes the word`sat,'' it is useful to know *who* sat (cat) and *where* (mat). Attention is a mechanism for a token to gather information from other tokens, weighted by relevance.

Similarity and Weighted Averaging

Relevance is measured by similarity between token representations. If token t and token s have similar representations, then s is likely relevant to t. We can formalize this:

1. Compute a *similarity score* between token t and every other token s in the sequence.
2. Convert scores to weights using softmax, so they sum to 1.
3. Compute a weighted average of all token representations, with higher weight on more relevant tokens.

The output for token t is not its original representation but a new vector that blends information from across the sequence, weighted by relevance. A token that is relevant to many others will contribute heavily to many output vectors.



"sat" attends to "cat" and "mat"

This is the attention idea: a dynamic, input-dependent routing of information across the sequence. The weights are not fixed — they are computed fresh for each input, which is what allows the model to resolve "it'' differently depending on context.

The Attention Formula

The attention mechanism is implemented through three learned linear projections of each token's representation: *query* (Q), *key* (K), and *value* (V).

For a sequence of T tokens with representation matrix $X \in \mathbb{R}^{T \times d}$, and projection matrices $W_Q, W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$:

$$Q = XW_Q \qquad K = XW_K \qquad V = XW_V$$

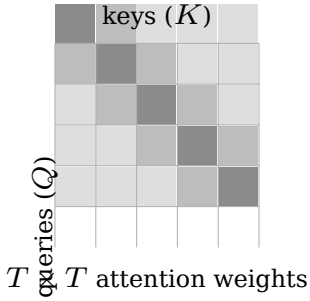
The query represents what a token is *looking for*. The key represents what a token *offers*. Relevance between token t and token s is the dot product $\mathbf{q}_t \cdot \mathbf{k}_s$.

Collecting all pairwise dot products into a $T \times T$ matrix and applying softmax row-wise gives the attention weights:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T \times T}$$

Entry A_{ts} is the weight that token t places on token s. The output is the weighted sum of value vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{T \times d_v}$$



Why Divide by $\sqrt{d_k}$?

Without scaling, the dot products $\mathbf{q}_t \cdot \mathbf{k}_s$ grow in magnitude with d_k — in high dimensions, dot products between random vectors have standard deviation $\sqrt{d_k}$. Large dot products push the softmax into saturation, producing near-zero gradients. Dividing by $\sqrt{d_k}$ keeps the inputs to softmax in a stable range regardless of embedding dimension.

Going deeper: the Q/K/V design. Why three separate projections rather than computing dot products directly on X? The projections allow the model to learn different notions of similarity for different tasks. W_Q and W_K transform the token representations into a space where dot products measure task-relevant similarity, rather than raw embedding similarity. W_V is separate because what a token *contributes* to others (its value) need not be the same as what it *advertises* (its key) or *seeks* (its query). The three-projection design gives the model three independent degrees of freedom per attention head.

Key result. Scaled dot-product attention computes, for each token, a weighted average of all other tokens' value vectors, where weights are determined by learned query-key similarity. The full formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The $T \times T$ weight matrix is computed fresh for every input — attention is data-dependent, not fixed.

Multi-Head Attention and the Transformer Block

Multi-Head Attention

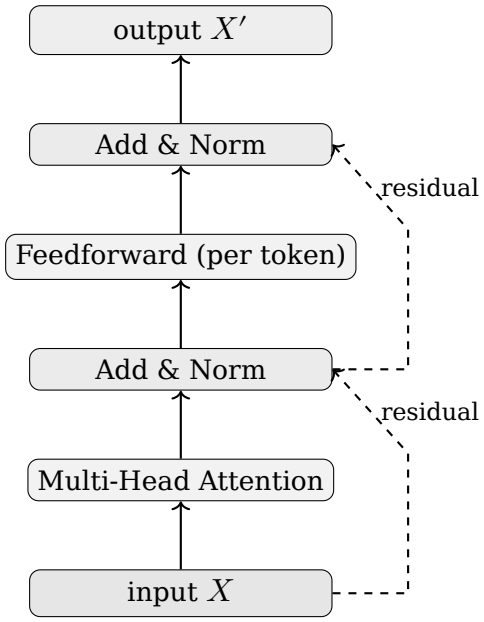
A single attention operation captures one kind of relationship between tokens. *Multi-head attention* runs h attention operations in parallel, each with its own projection matrices, then concatenates the results:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

where each $\text{head}_i = \text{Attention}(XW_{Q_i}, XW_{K_i}, XW_{V_i})$ and $W_O \in \mathbb{R}^{hd_v \times d}$ is an output projection. Different heads learn to attend to different kinds of relationships simultaneously — syntactic structure, coreference, semantic similarity — within the same layer.

The Transformer Block

One transformer block applies two sublayers to the sequence, each followed by a residual connection and layer normalization:



The residual connections ("Add'' in Add & Norm) add the block's input directly to its output before normalization. This gives gradients a direct path back through many layers during backpropagation, and is one of the key reasons transformers can be trained very deep.

Going deeper: why residual connections help. Without residual connections, the gradient of the loss with respect to early-layer parameters must pass through every intermediate layer — and if any layer's Jacobian has small singular values, the gradient shrinks exponentially with depth. A residual connection adds an identity path: output $= F(x) + x$, so $\frac{\partial \text{output}}{\partial x} = \frac{\partial F}{\partial x} + I$. The identity term ensures gradients are at least as large as the identity matrix, preventing vanishing gradients regardless of what F learns. This insight, from He et al. (2015), enabled networks of 100+ layers and directly influenced the transformer design.

Parameters in One Transformer Block

For a model with embedding dimension d and h attention heads (each with $d_k = d_v = d/h$):

Component	**Parameters**
W_Q, W_K, W_V (per head, $\times h$)	$3 \times d \times d$
Output projection W_O	$d \times d$
Feedforward layer (expand $\rightarrow$ contract)	$\approx 8d^2$
Layer norm parameters (2 sublayers)	$4d$
Total per block	$\approx 12d^2$

For GPT-3 with $d = 12{,}288$: each block has approximately $12 \times 12288^2 \approx 1.8$ billion parameters. GPT-3 has 96 blocks.

From Block to Model

A transformer language model stacks N identical blocks, preceded by the embedding layer and followed by an output head:

$$\text{input tokens} \xrightarrow{\text{embed + pos}} X^{(0)} \xrightarrow{\text{block 1}} X^{(1)} \xrightarrow{\text{block 2}} \cdots \xrightarrow{\text{block } N} X^{(N)} \xrightarrow{\text{output head}} \text{logits}$$

The output head is a linear projection $X^{(N)}W_{\text{out}}$ where $W_{\text{out}} \in \mathbb{R}^{d \times V}$ maps from embedding dimension back to vocabulary size. Applying softmax to the logits gives a probability distribution over the vocabulary.

The GPT-3 Parameter Count

Component	**Parameters**
Token embedding matrix (50257×12288)	≈ 617M
Positional embedding (2048×12288)	≈ 25M
96 transformer blocks (≈ 1.8B each)	≈ 173B
Output head	shared with embedding
Total	≈ 175B

Every one of these parameters is updated by gradient descent. Every gradient is computed by backpropagation. The update rule is $\theta \leftarrow \theta - \alpha \nabla_\theta L$, unchanged from the two-parameter linear regression we started with.

What Large Language Models Actually Do

A language model is trained on a single objective: given a sequence of tokens, predict the next one. The loss is cross-entropy between the model's predicted distribution over vocabulary and the true next token. This is the same cross-entropy from Handout 1.

At inference, the model generates text by sampling repeatedly from its output distribution: given all tokens so far, produce a probability distribution over the next token, sample from it, append the result, and repeat.

Common misconception. A language model does not retrieve text from a database. It does not look up answers. It computes a probability distribution over possible next tokens given the input, and samples from that distribution. The "knowledge" in the model is not stored in any retrievable form — it is distributed across 175 billion parameter values that were shaped by gradient descent on the training data. This is why models can be wrong, why they can hallucinate plausible-sounding nonsense, and why their errors are so different from the errors a search engine would make.

Key result. A large language model is a feedforward computation (embedding $\to$ N transformer blocks $\to$ softmax) trained by gradient descent to minimize cross-entropy loss on next-token prediction. The mathematics is: linear algebra for the forward pass, probability theory for the loss function, and calculus — specifically the chain rule via backpropagation — for learning. These are not analogies. They are the actual operations the model performs.

The Full Parameter Scaling Story

Model	Parameters	New idea
Linear regression	2	Loss + gradient descent
Single neuron	$d + 1$	Nonlinear activation
Feedforward network	$O(nm)$/layer	Composition + backprop
Transformer block	$\approx 12d^2$	Attention mechanism
GPT-3	175 billion	Scale

Each row adds one idea. The learning algorithm never changes. The chain rule, which your calculus students encounter in week four of Calc I, is the engine running underneath all of it.